

University of Science and Technology Chittagong (USTC)

Department of Computer Science and Engineering



Lab Report-1

Course Title: Computer Graphics and Image Processing Lab

Course Code: CSE 418

Submitted To

Umme Aiman Jannat

Lecturer

CSE, FSET, USTC

Submitted By

Name: **Zahid Al Noor Himanta**

Roll No: **23070152**

Registration No: **1058**

Batch: **41 (Section- B, CSE)**

7th Semester

Experiment 1: Implementation of Translation of Image

Source Code:

```
[1] import cv2 #OpenCV library for python
    import numpy as np
    import matplotlib.pyplot as plt

[2] image = cv2.imread('/content/ninja_h2r.png')
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

[3] #image dimension

    h, w = image.shape[:2]

[4] #translation values

    tx = 100
    ty = 50

[5] #Tranlation matrix

    M = np.float32(
        [[1, 0, tx], [0, 1, ty]]
    )

[6] #Apply translation

    translated = cv2.warpAffine(image, M, (w, h))

[7] #Display

    plt.figure(figsize = (10, 5))

    ... <Figure size 1000x500 with 0 Axes>
        <Figure size 1000x500 with 0 Axes>

[8] plt.subplot(1, 2, 1)
    plt.imshow(image)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(translated)
    plt.title("Translated Image")
    plt.axis("off")

    plt.show()
```

Output:

Original Image



Translated Image



Experiment 2: Implementation of Scaling of Image

Source Code:

```
[ ] import cv2 #OpenCV library for python
import numpy as np
import matplotlib.pyplot as plt

[ ] image = cv2.imread('/content/Kawasaki-H2-Carbon-New.png')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

[ ] #image dimension

h, w = image.shape[:2]

[ ] #scaling factors

sx = 4
sy = 3

[ ] #scaling matrix

M = np.float32(
    [[sx, 0, 0],
     [0, sy, 0]]
)

[ ] scaled = cv2.warpAffine(
    image,
    M,
    (int(w*sx), int(h*sy))
)

[ ] plt.figure(figsize = (10, 5))

<Figure size 1000x500 with 0 Axes>
<Figure size 1000x500 with 0 Axes>

[ ] plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(scaled)
plt.title("Scaled Image")
plt.axis("off")

plt.show()
```

Output:

Original Image



Scaled Image



Experiment 3: Implementation of Rotation of Image.

Source Code:

```
[1] import cv2 #OpenCV library for python
✓ 0s import numpy as np
import matplotlib.pyplot as plt

[2] image = cv2.imread('/content/ninja_h2r.png')
✓ 0s image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

[3] #image dimension
✓ 0s h, w = image.shape[:2]

[4] #center of image
✓ 0s center = (w// 2, h//2)

[5] #Rotation angle
✓ 0s angle = 90

[6] #Rotation matrix
✓ 0s M = cv2.getRotationMatrix2D(center, angle, 1.0)

[7] rotated = cv2.warpAffine(image, M, (w, h))
✓ 0s

[8] plt.figure(figsize= (10, 5))
✓ 0s
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(rotated)
plt.title("Rotated Image")
plt.axis("off")

plt.show()
```

Output:

Original Image



Rotated Image



Experiment 4: Implementation of Reflection of Image in X and Y axis.

Source Code:

```
[1]
✓ 0s  import cv2  #OpenCV library for python
      import numpy as np
      import matplotlib.pyplot as plt

[2]
✓ 0s  image = cv2.imread('/content/ninja_h2r.png')
      image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

[3]
✓ 0s  #Reflection about X-axis

      reflected_x = cv2.flip(image, 0)
      reflected_y = cv2.flip(image, 1)

[4]
✓ 2s  plt.figure(figsize= (25, 20))

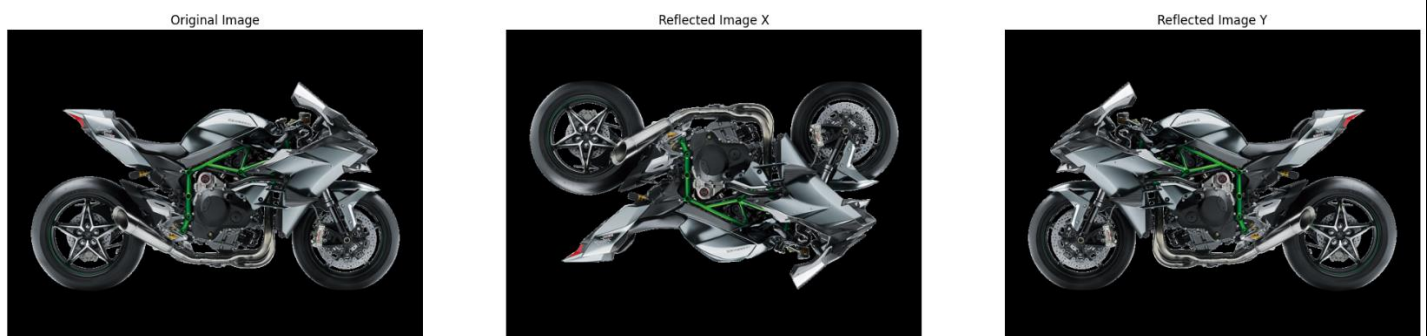
      plt.subplot(1, 3, 1)
      plt.imshow(image)
      plt.title("Original Image")
      plt.axis("off")

      plt.subplot(1, 3, 2)
      plt.imshow(reflected_x)
      plt.title("Reflected Image X")
      plt.axis("off")

      plt.subplot(1, 3, 3)
      plt.imshow(reflected_y)
      plt.title("Reflected Image Y")
      plt.axis("off")

      plt.show()
```

Output:



Experiment 5: Implementation of Translation, Scaling, Rotation and Reflection of Image.

Source Code:

```
[1] import cv2 #OpenCV library for python
✓ 0s import numpy as np
import matplotlib.pyplot as plt
```

```
[2] image = cv2.imread('/content/ninja_h2r.png')
✓ 0s image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
[3] #image dimension
✓ 0s
h, w = image.shape[:2]
```

Step-1: Scaling

```
[4] sx = 10
✓ 0s sy = 7

scaled = cv2.resize(image, (0,0), fx=sx, fy=sy)
```

Step-2: Rotation

```
[5] h2, w2 = scaled.shape[:2]
✓ 0s

[6] center = (w2//2, h2//2)
✓ 0s

[7] M_rotation = cv2.getRotationMatrix2D(center, 45, 1)
✓ 0s

[8] rotated = cv2.warpAffine(scaled, M_rotation, (w2, h2))
✓ 0s
```

Step-3: Translation

```
[9] tx = 50
✓ 0s ty = 30

[10] M_translation = np.float32([[1, 0, tx], [0, 1, ty]])
✓ 0s

[11] transformed = cv2.warpAffine(rotated, M_translation, (w2+tx, h2+ty))
✓ 0s
```

Step-4: Reflection

[12]
✓ 0s

```
#Reflection about X-axis
```

```
reflected_x = cv2.flip(image, 0)  
reflected_y = cv2.flip(image, 1)
```

[13]
✓ 7s

```
#Display All
```

```
plt.figure(figsize= (10, 10))
```

```
plt.subplot(2, 3, 1)  
plt.imshow(image)  
plt.title("Original Image")  
plt.axis("off")
```

```
plt.subplot(2, 3, 2)  
plt.imshow(scaled)  
plt.title("Scaled Image")  
plt.axis("off")
```

```
plt.subplot(2, 3, 3)  
plt.imshow(rotated)  
plt.title("Rotated Image")  
plt.axis("off")
```

```
plt.subplot(1, 3, 1)  
plt.imshow(transformed)  
plt.title("Transformed Image")  
plt.axis("off")
```

```
plt.subplot(1, 3, 2)  
plt.imshow(reflected_x)  
plt.title("Reflected Image X")  
plt.axis("off")
```

```
plt.subplot(1, 3, 3)  
plt.imshow(reflected_y)  
plt.title("Reflected Image Y")  
plt.axis("off")
```

```
plt.show()
```

Output:

Original Image



Scaled Image



Rotated Image



Transformed Image



Reflected Image X



Reflected Image Y

